

VI. SUPPLEMENTARY

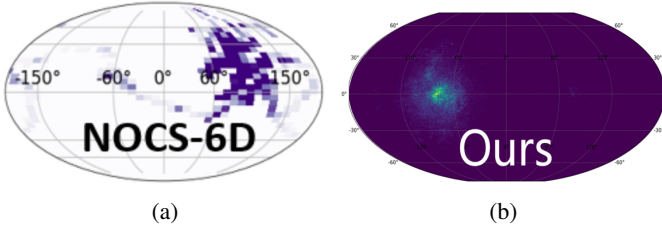


Fig. 4: Viewpoint coverage map. Most of our benchmarked COPE algorithm is trained on NOCS-6D dataset. Differ from NOCS-6D, our viewpoint coverage is more balanced between top-down and bottom-up views. Note that the left figure is referred to [18]

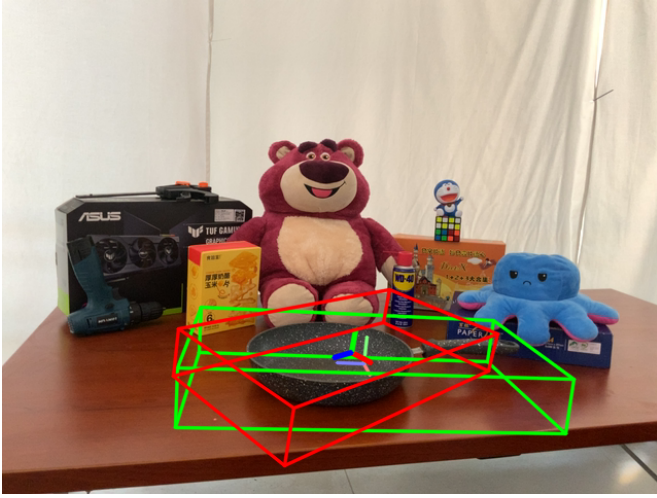


Fig. 5: Foundpose failure case

1) *UOPE*: This algorithm heavily relies on features extracted by DINOv2 to establish 2D-3D correspondences, which makes it susceptible to pose estimation errors for low-texture or highly symmetric objects. As illustrated in the Fig.5, the object (a frying pan) has relatively sparse texture, and during the detection process, the right-side handle was mistakenly ignored. As a result, the object was erroneously treated as symmetric, leading to an incorrect pose estimation.

We directly employ the model-based FoundationPose, whose object pose estimation primarily relies on the input RGB-D data. When the camera undergoes rapid motion, causing motion blur between frames, the pose estimation produced by FoundationPose becomes error-prone as well. This issue is especially prominent for flat, textureless, and symmetric objects, where the system may incorrectly estimate the orientation of the coordinate axes. An example failure case is shown in Fig.6

The performance of GigaPose benefits from its time-intensive pre-processing step, where it renders the input CAD model extensively before performing pose estimation. This process allows it to establish more accurate correspondences during object pose estimation. However, since the method



Fig. 6: Fdpose failure case

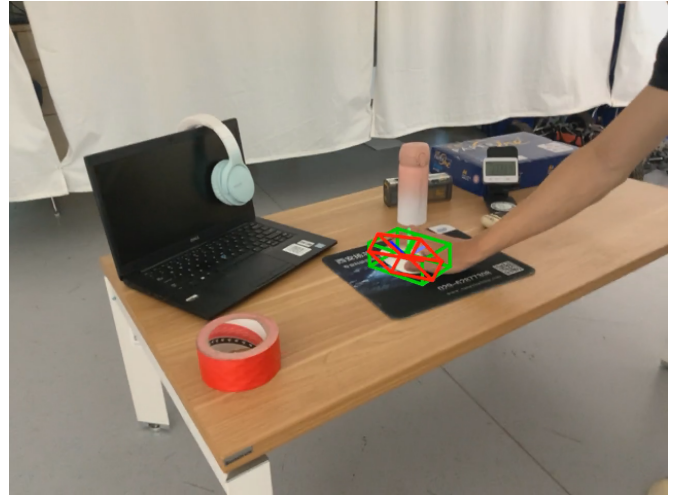
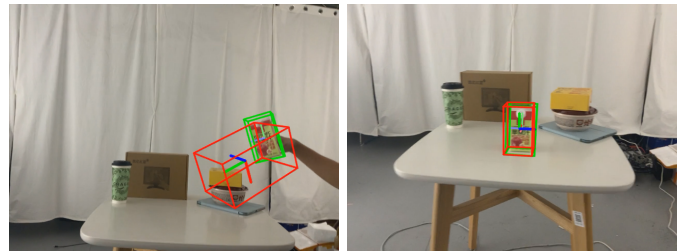


Fig. 7: Gigapose failure case

relies solely on RGB information and performs feature matching across different viewpoints of the CAD model, it faces significant challenges when dealing with occlusions or low-texture objects. As shown in Fig.7, such limitations result in a failure case where the person’s hand holding the mouse causes the estimation to break down.



(a) Mega(RGB) failure case (b) Mega(RGB-D) failure case

Fig. 8: Megapose failure cases

As shown in Fig.8, here we evaluate both the RGB-based and RGB-D-based variants of MegaPose. This method takes

a cropped image (RGB or RGB-D) as input along with the corresponding CAD model. Multiple input poses are rendered from the CAD model and then passed to the proposed coarse estimator to obtain initial poses, which are subsequently refined by a refinement network to produce the final output. It is important to note that the final results heavily depend on the coarse network—once the initial pose estimation is inaccurate, the refinement stage is generally unable to recover the correct poses. Therefore, both RGB and RGB-D approaches are highly dependent on the quality of the multiple input poses. To ensure a fair comparison, we use the 2D bounding box derived from the ground-truth mask as the input for crop. For geometrically symmetric or regular-shaped objects such as the juice box shown, the use of multiple input poses can easily lead to incorrect initial guesses. Consequently, failure cases are observed in both the RGB-based and RGB-D-based versions of the method, as illustrated in the figure.

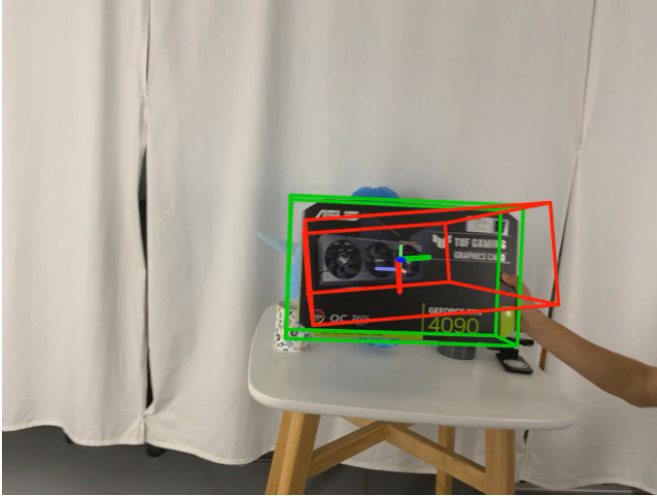


Fig. 9: SAM6D failure case

The core idea of SAM6D is to first segment any object in the scene using SAM, and then match the segmented mask to the target object. Finally, a GeoTransformer—a learning-based point matching method—is applied to estimate the object’s pose. Therefore, SAM6D is inherently a multi-stage pipeline. For symmetric, textureless, or geometrically simple objects, errors often occur in the final pose estimation due to incorrect registration, ultimately resulting in failure cases such as the one shown in Fig.9

2) *OPT*: For MaskFusion, to ensure a fair benchmark, we replaced its Mask R-CNN module with masks generated by SAM2. However, when significant relative motion occurs—especially simultaneous movement of the camera and the object—the pose estimation of the object in consecutive frames is often mistakenly identified as tracking a new object. Consequently, this leads to noticeable drift in the object pose estimation over the course of the sequence. An example failure case is shown in Fig.10

BundleTrack takes as input an RGB-D sequence along with object segmentation results. It then uses traditional keypoint matching methods to estimate the relative poses between consecutive frames, which are subsequently optimized through



Fig. 10: Maskfusion failure case

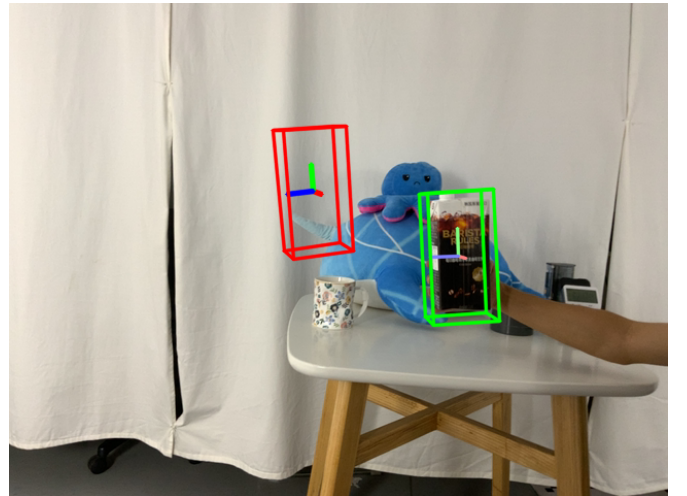


Fig. 11: Bundlesdf failure case

an object pose graph. The pipeline is straightforward and clear. However, as Fig.11 shows, its main drawback is that for objects with sparse texture, the relative pose estimations often become inaccurate, leading to drift accumulations in subsequent frames.

Bundlesdf(Fig.12) demonstrates overall robust performance without significant drift or failure cases. However, it still faces considerable challenges when dealing with low-texture and highly symmetric objects. Moreover, as Bundlesdf is a Neural Object Field-based approach, it simultaneously estimates the object’s 6DoF pose and reconstructs the object’s mesh. Consequently, the runtime for processing an entire sequence is considerably limited.

DROID-SLAM, as a popular SLAM framework, takes RGB-D inputs and was originally proposed for camera pose tracking and scene reconstruction. When provided with segmented images and fed into its DBA layer, it operates in an object-centric setting, allowing us to obtain the object pose for each frame. However, since the masked objects provide limited input information, any error in camera pose estimation can lead to significant drift(Fig.13).



Fig. 12: BundleSDF failure case

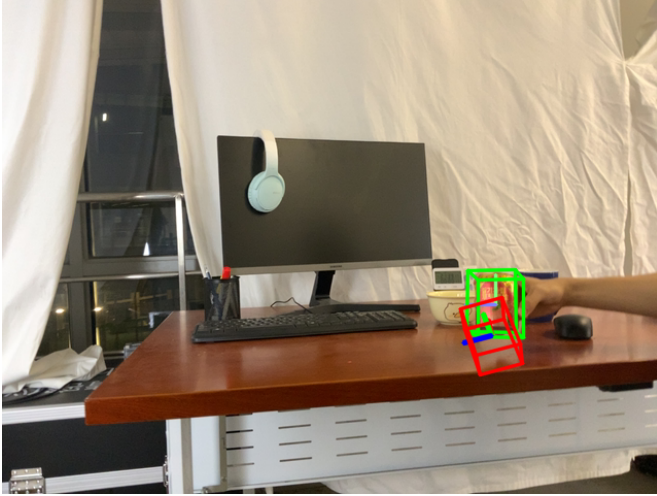


Fig. 13: Droid-SLAM failure case

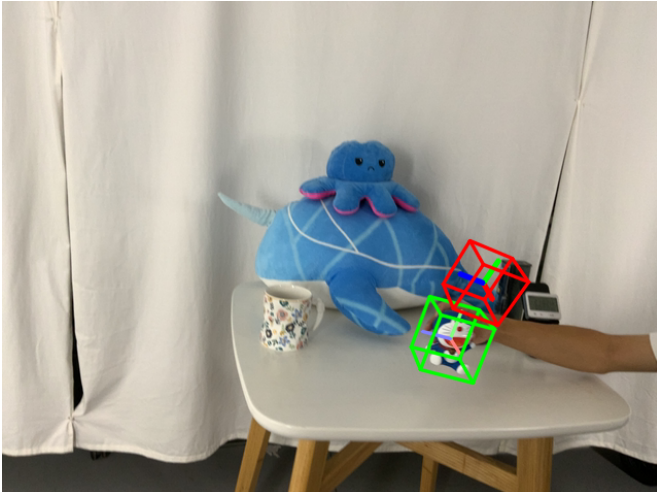


Fig. 14: Fdpose-track failure case

The tracking component of FoundationPose experiences drift and tracking loss when subjected to rapid motion, especially in scenarios involving simultaneous movement of both the camera and the object. At certain moments, the relative motion speed becomes large enough to cause tracking failures (Fig. 14), thereby confirming that our dataset presents significant challenges to state-of-the-art algorithms.

3) *COPE*: NOCS was the first to propose using a normalized object coordinate space to estimate the pose of objects at category level. It first employs a NOCS network to predict object points and then applies the Umeyama algorithm [28] to recover the object's pose and size. Since NOCS performs object detection and point prediction jointly, for fairness in benchmarking, we also adopt the settings from MV-ROPE, where consistent segmentation results (shared with other methods) are provided as input. It can be observed that even in cases where NOCS object detection is successful, the method has clear limitations. The object pose estimation relies entirely on the quality of the predicted points in NOCS space. Furthermore, because the Umeyama algorithm is used directly, when the predicted points—either from the original NOCS predictor or the MV-ROPE implementation—are of poor quality (as shown for the pink bottle in the Fig 15), recovering an accurate pose and size becomes extremely difficult.

Overall, in these challenging cases, methods that rely on predicting a NOCS map—such as [9], [32] and [49]—followed by the Umeyama algorithm to compute the object pose, tend to exhibit significantly higher translation errors compared to methods that directly predict translation, such as [35], [37], [39], [40], [41], [50]. This performance gap can be attributed to the fact that most of the latter methods, including [35], [39], [40], [41], implement translation prediction using PointNet-based architectures and decouple it from rotation prediction. In the COPE task, compared to rotation estimation, translation prediction techniques are relatively more mature and stable.

We also conducted a focused analysis of the failure cases encountered by the evaluated COPE algorithms on our dataset. Notably, the most prominent failure cases fall into two main categories: the camera category and the mug category.

- 1) The camera category has long been a known challenge in COPE tasks, primarily due to the large intra-class variation among commonly seen camera models. Cameras in real-world scenarios exhibit a wide range of appearances, posing significant challenges for the generalization ability of pose estimation networks.
- 2) The mug category, by contrast, tends to have more consistent geometric shapes. However, failure cases often occur when a mug is being held by a human hand. Such occlusion frequently leads to errors in rotation estimation. Another possible cause of failure is when the mug's opening is not visible in the input image—under such conditions, it becomes difficult for the network to distinguish the upright orientation of the mug.

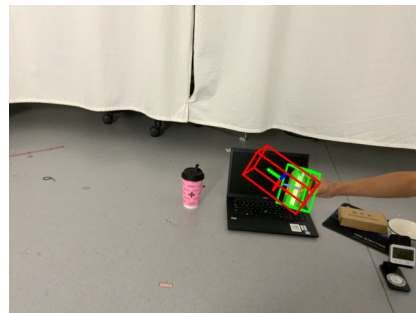
4) *finetune*: Moreover, under our Dynamic Object with Moving Camera scenario, the dataset presents significantly different challenges compared to static real-world datasets such as REAL275 or HouseCat6D. Our dataset captures novel viewpoints induced by object rotations along different axes



(a) NOCS



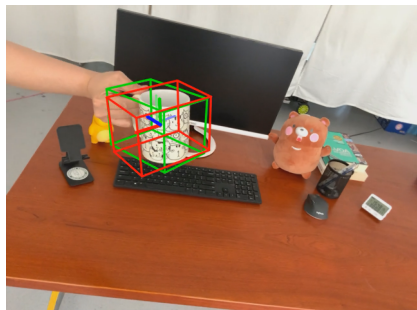
(b) NOCS(MV-ROPE)



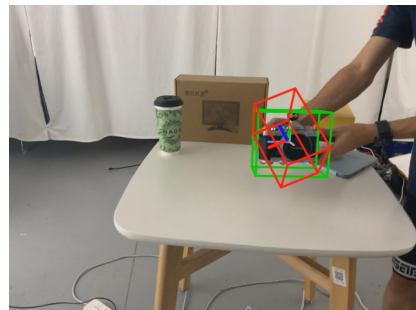
(c) Diffusion-NOCS



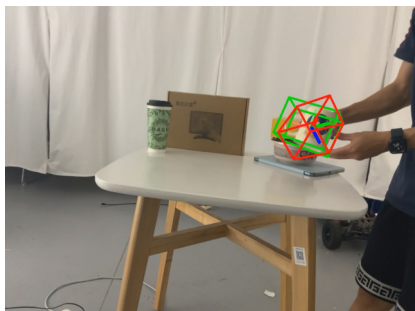
(d) Agpose



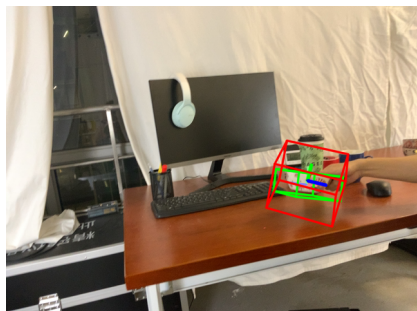
(e) Second-pose



(f) IST-Net



(g) VI-Net

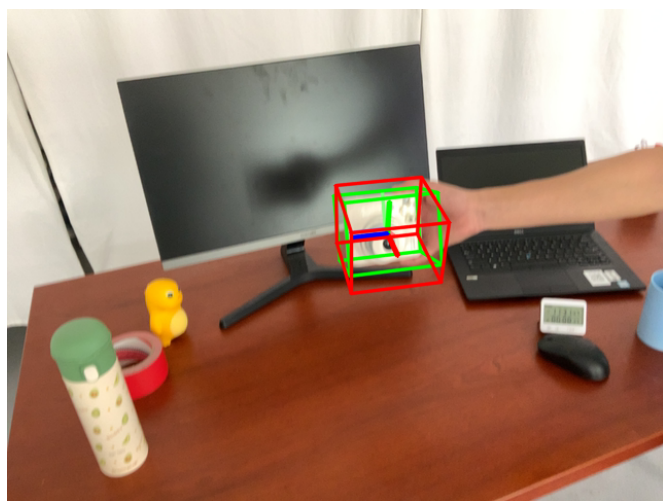


(h) GCASP



(i) Gen-pose

Fig. 15: Nine example cases



(a) Agpose-origianl

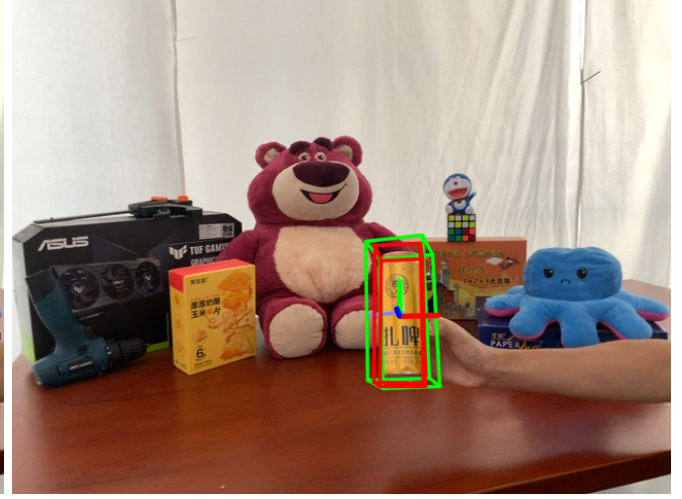


(b) Agpose-finetuned

Fig. 16: Fintune-agpose



(a) Secondpose-original



(b) Secondpose-finetune

Fig. 17: Fintune-secondpose

while being held in hand—viewpoints that are difficult to observe in static scenes. These challenges manifest in pose estimation failures for various methods, such as GCASP when a bowl is lifted upward, AGPose and GenPose when the camera is moved upward, and DiffusionNOCS when a can is lifted vertically. Furthermore, in our fine-tuning experiments (see Section III), it is evident that fine-tuning on our dataset leads to substantial improvements. For example, in Fig. 16 and Fig. 17, when comparing SecondPose and AGPose before and after fine-tuning, the previously mentioned pose estimation failures caused by vertical lifting of objects are significantly mitigated.